

Agent 总在打转？从 SFT 到 Harness，后训练怎么做

nano
江苏科技大学 信号与信息处理硕士
关注

假设一个 Agent 在单轮评测里接近满分，上线做多轮任务却反复读同一份文件：第 7 轮调用 `read_file("config.yaml")`，第 8 轮读到相同内容，第 9 轮还在读。demo 里三步完成的任务，到了真实循环里就停不下来。

这种故障该修哪里？我的判断是，先排查 Harness，再决定后训练补什么能力。只看最终答案的评测，会漏掉中间的重复动作；直接加训练，也可能把上下文组织的问题带进训练环境。

先沿着一次工具调用查三处。第一，Harness 回填的工具结果是否让下一轮上下文始终没有变化。第二，第 7 轮附近有没有发生截断，把前面已经完成的进展挤出窗口。第三，循环保护怎样处理相同调用：如果“连续两次相同调用就提醒”，提醒会不会继续触发下一轮。

还可以在回放中做一次提示反事实。在第 7 轮的 observation 后补上“你已经读过 config.yaml，内容没变，下一步应该修改它”，再跑同一条任务。如果动作改变，说明提示组织值得继续查；如果动作没变，也要结合更多回放定位。这一次干预提供的是线索，不能单凭它给故障定因。

确认问题来自模型的决策策略，再考虑后训练。多轮任务要学的是一串选择怎样衔接。假设轨迹长 12 轮，终点评测只能告诉你最后做没做成，前面 11 轮的动作质量还得另看。

例如，两条轨迹最终都答对，终点都拿 1.0。一条走 `read_file` → 修改配置 → 运行测试，另一条猜三次才碰到正确答案。只给终点奖励，两条路径拿到同样的分数，猜测动作也可能被一起强化。训练要区分它们，需要轨迹数据、判分规则与多轮评测共同配合。

先分清：你要训练模型的哪一段行为？

Agent = Model + Harness。 Harness 是模型外的执行系统，负责调用模型、处理工具调用和决定何时停止。Scaffold 的范围更窄，通常指 system prompt、工具描述、响应解析和跨步记忆。

做产品时，这些词对应团队接口。工具描述、记忆和停止规则放在哪里，决定了训练能复现多少线上行为；写训练代码时，这也是 `environment_factory` 要明确接住的边界。

TRL 提供的五个位置，可以按学习信号来理解：

▲ 赞同 ▼ 添加评论 5 喜欢 分享 申请转载 ···

研究 AI 算法与工程实践，...
江苏科技大学 信号与信息处理硕士
回答 400 文章 575 关注者 85
关注 发私信
大家都在搜 换一换
陕西发生重大刑案嫌疑人持... 653 万 热
张雪重庆大学开价被 识图提问 万 热
南京大学抬头率 420 万 热

	assistant token		数据列
蒸馏	教师逐 token 的软分布	DistillationTrainer	prompt 数据、教师模型
GRPO	每个答案的奖励分数	GRPOTrainer	prompt、奖励函数
RL 环境	动作之后的观察与结果	GRPOTrainer	任务、环境，可接 OpenEnv
Harness 与扩展	完整 Agent run 的轨迹	AsyncGRPOTrainer	任务、环境、线上 Harness

SFT 的标签明确，蒸馏进一步提供下一 token 的概率分布；到了结果奖励，反馈会稀疏得多。监督越细，越容易知道该教哪个动作，也越依赖示范者；环境越接近部署，越能暴露真实故障，排错和采样的成本也越高。

这五个入口允许按任务选择。已有正确轨迹，只是某个动作没学好，可以先做 SFT 和多轮评测；要让模型在自己生成的前缀上得到纠正，可以考虑蒸馏；能可靠计算结果分数，可以用 GRPO。重复调用若来自回填或截断，就先修 Harness。

另一个分界是状态。单次调用相互独立，用 `tools`；动作会改变下一轮观察，而且状态要跨轮保留，用 `environments`。浏览、导航和改代码后跑测试，通常需要后一种建模。

SFT：已有正确轨迹，先教会它照着走

SFT 适合手里已有高质量多轮轨迹、目标是让模型掌握已知正确动作的情况。它像临摹字帖：训练给出写好的路径，模型学习每一步怎样接下去。路径中没有包含的偏离与恢复，需要另外补数据或信号。

对不准备写训练脚本的读者，判断到这里就够：数据里有没有成功路径，往往比先选哪个训练参数更值得问。

落到实现，先看 `assistant_only_loss` 和 `packing`。前者把损失限制在 assistant 回合，system 与 user 位置用 -100 屏蔽；后者把短样本装进同一序列，减少 padding。损失掩码还依赖模板，不能只看开关名字。

下面这条样本同时保留工具定义、工具调用、工具返回和最终回答。先打印一条数据，检查 `messages` 的层级与 `tools` 的 schema，确认工具定义完整，再调超参。

解释

```
{
  "messages": [
    {"role": "system", "content": "你是一个只读文件助手。需要文件内容时调用 read_file,
    {"role": "user", "content": "config.yaml 里的 timeout 是多少？"},
    {"role": "assistant", "content": "", "tool_calls": [{"id": "call_1", "type":
    {"role": "tool", "tool_call_id": "call_1", "content": "timeout: 30\nretries:
    {"role": "assistant", "content": "timeout 是 30 秒。"}
  ],
  "tools": [
    {
      "type": "function",
      "function": {
        "name": "read_file",
        "description": "读取工作区内的文本文件并返回全文。",
        "parameters": {
          "type": "object",
          "properties": {"path": {"type": "string", "description": "相对工作区根目录
          "required": ["path"]
        }
      }
    }
```

`arguments` 在这个示例里用对象。Qwen2.5 模板会对参数做 `tojson`，直接存 JSON 字符串可能再转义一次，导致调用格式变样。收到服务端返回的字符串参数时，先 `json.loads` 转成对象再落数据。

保存为 `traces.jsonl`，每行一条完整轨迹，先积累几十条作为起步数据。下面用 `json.loads` 后接 `Dataset.from_list`。`tools` 保持结构化列表，减少隐式转换；虽然相关源码会处理字符串形式，也不应依赖这层兜底来定义数据协议。

多条轨迹的工具 schema 不一致时，还要检查 Datasets 的类型推断。它可能把不同对象合并成同一结构；可为该列显式指定 `Json()` 特征、用 `on_mixed_types="use_json"`，或通过 `Features` 固定类型。

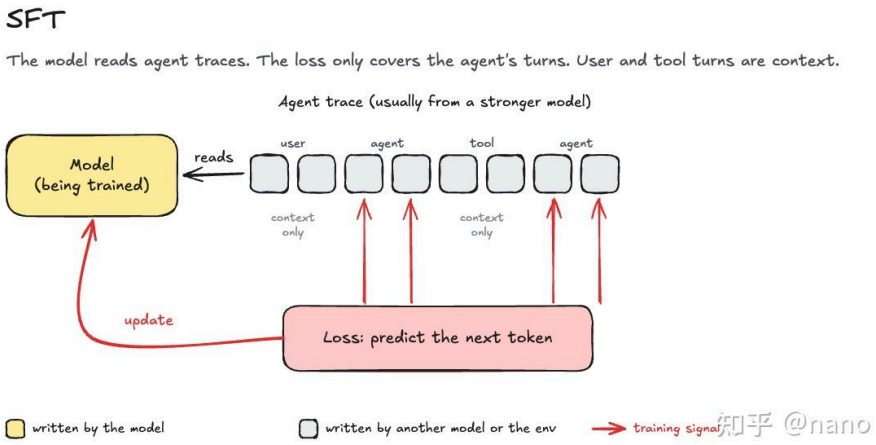
解释

示例按 TRL v1.17.1 写成。为了复现同一接口，先固定版本；切换新版本时逐项对照文档。

```
pip install -U "trl==1.17.1" transformers datasets accelerate
```

显存没有跨配置通用的“够用”数字。batch size、序列长度、优化器状态与 LoRA 都会改变占用。先选 0.5B 级模型，从 2048 到 4096 的序列长度起步，确认数据与训练链路，再增加规模；同样条件下，1B 全参更容易遇到 OOM。

显存紧时先降 `max_length`，再降 batch。下面默认关闭 packing，减少首次接线的依赖；确认 FlashAttention 等条件后再开。packing 的主要收益来自减少 padding，也要核对实际打包长度和样本边界。



Hugging Face 原图：灰色用户与工具回合只作上下文，红箭头把监督落在 Agent token 上。

解释

```python
import json
from datasets import Dataset
from trl import SFTConfig, SFTTrainer

# traces.jsonl 的每行是一条完整样本，字段与上面的 JSON 一致
dataset = Dataset.from_list(
    [json.loads(line) for line in open("traces.jsonl", encoding="utf-8")]
)

trainer = SFTTrainer(
    model="Qwen/Qwen2.5-0.5B-Instruct",
    train_dataset=dataset,
    args=SFTConfig(
        assistant_only_loss=Tru
        packing=False,
```

```
)
trainer.train()
```

训练前有四项检查：

1. 每条数据都有 `messages` 与结构化 `tools` 。
2. 抽样打印 loss mask，system/user 位置确实为 -100。
3. chat template 带 generation 掩码， `assistant_only_loss` 生效。TRL 会为 Qwen2.5 等已知家族使用相应训练模板，仍须核对实际渲染。
4. 训练前后使用同一套多轮评测，比较整条任务的行为。

评测最好复用线上 Harness。 `eval_multiturn.py` 只负责加载 checkpoint、读取任务，再把 policy 交给原来的 tool loop。下面的 `your_harness` 是你需要替换的集成接口，返回文本的 policy 也需要由该 Harness 解析工具调用。

解释

```python
# eval_multiturn.py：薄薄一层包装，复用线上同一个 harness
import argparse, json
from transformers import AutoModelForCausalLM, AutoTokenizer
from your_harness import run_task  # 换成线上 tool loop 的模块名

ap = argparse.ArgumentParser()
ap.add_argument("--model", required=True)
ap.add_argument("--tasks", required=True)
ap.add_argument("--max-turns", type=int, default=20)
ap.add_argument("--report", required=True)
args = ap.parse_args()

tok = AutoTokenizer.from_pretrained(args.model)
model = AutoModelForCausalLM.from_pretrained(args.model, device_map="auto")

def policy(messages, tools):
    ids = tok.apply_chat_template(messages, tools=tools, add_generation_prompt=Tru
                                  return_tensors="pt").to(model.device)
    out = model.generate(ids, max_new_tokens=512, do_sample=False)
    return tok.decode(out[0][ids.shape[-1]:], skip_special_tokens=True)

rows = []
for line in open(args.tasks, encoding="utf-8"):
    task = json.loads(line)
    rows.append({"task_id": task["id"], **run_task(policy, task, max_turns=args.ma

json.dump(rows, open(args.report, "w", encoding="utf-8"), ensure_ascii=False, inde
```

`tasks.jsonl` 每行至少包含 `id` 、 `messages` 、 `tools` 。约定 `run_task` 返回 `turns` 、 `repeated_calls` 、 `stop_reason` 与最终 `answer` ；停止原因可区分 `model_stop` 、 `max_turns` 、 `tool_error` 。这些字段的数组写进 `out/sft-eval.json` ，才能比较模型答对之前绕了多少路。

几个常见故障可以沿数据往回查：

1. 缺 `tools` 列时，模板拿不到工具定义，模型可能学成纯文本模拟调用。逐行查列是否齐全。
2. 参数被存成字符串时，检查是否发生二次转义，先转对象再进入模板。
3. 开 `assistant_only_loss` 后提示模板不支持时，核对 `{% generation %}` 标记；临时关闭开关可帮助排查数据，但正式训
4. loss 下降、线上表现变差时，检

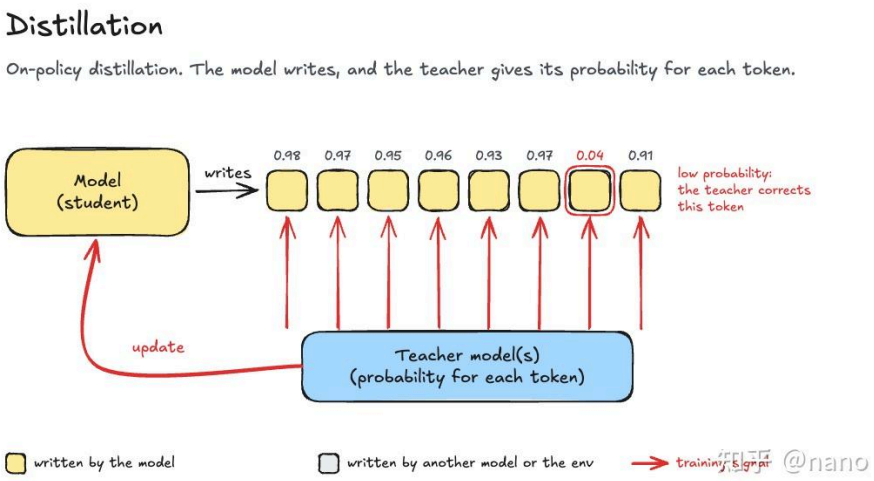
蒸馏：学生走出的前缀，也能得到教师反馈

成功轨迹不够多、却有更强的可用模型时，可以考虑蒸馏。它多提供一层信息：学生自己生成前缀，教师在同一个前缀上给出下一 token 的分布，学生学习对齐。即使前缀已经偏离示范，教师仍能对这个局面给反馈。

纯文本任务可以只准备 prompt；工具任务还需要向 Trainer 提供与线上一致的同步工具函数。只有 prompt 数据，并不等于已经复现了 Agent 环境。

这里重点看 `beta`。它控制广义 JSD 的插值：靠近 0 时趋向 forward KL，偏向覆盖教师分布；靠近 1 时趋向 reverse KL，偏向教师概率较高的区域。中间值用来平衡两种倾向，实际效果仍要看任务和生成长度。

vLLM 可以加速学生侧的 on-policy 生成，也就是用学生当前策略生成训练所需的前缀。教师对同一前缀做分布匹配，额外增加前向计算；两部分耗时最好分别记录。



Hugging Face 原图：学生先生成，教师对同一前缀提供逐 token 概率；红框标出低概率 token。

```python
from datasets import load_dataset
from trl import DistillationConfig, DistillationTrainer

# 官方 prompt-only 数据集：每行只需要 prompt，不需要参考答案
dataset = load_dataset("trl-lib/ultrafeedback-prompt", split="train")

trainer = DistillationTrainer(
    model="Qwen/Qwen2.5-0.5B-Instruct",
    teacher_model="Qwen/Qwen2.5-1.5B-Instruct",
    train_dataset=dataset,
    args=DistillationConfig(
        beta=0.5,              # JSD 插值：0 偏覆盖，1 偏取模
        max_completion_length=8192,  # 学生每次生成的上限；默认 512
        output_dir="out/distill",
    ),
)
trainer.train()
```

旧代码若使用 `GKDTrainer`⁺，其导入位置是 `trl.experimental.gkd`；新代码可以按上面的 `DistillationTrainer` 接口编写。

接线时检查四件事：

2. 学生与教师的 chat template 保持一致。
3. 先抽 20 条看学生生成长度分布，记录教师前向的耗时与覆盖情况。
4. 分别统计学生生成和教师前向占总 step 的比例，找到实际瓶颈。

教师与学生同卡 OOM 时，可以先考虑把教师移到另一张卡，或采用适用的量化方案，再判断是否要改 batch。学生回复持续变短，尤其在 `beta` 靠近 1 时，要检查 EOS、`max_completion_length` 与分布匹配的影响，可尝试把 beta 调回 0.3 到 0.5 观察变化。

这里没有“跑到固定步数必然塌缩”的结论。短回答退化是一种需要监控的风险，是否发生取决于具体训练。

蒸馏的代价是教师前向和模型资源开销，教师能力也会限制监督质量。总成本取决于部署与实现，不能直接按“推理账单翻倍”估算。

GRPO：判得出对错，就用结果分数训练

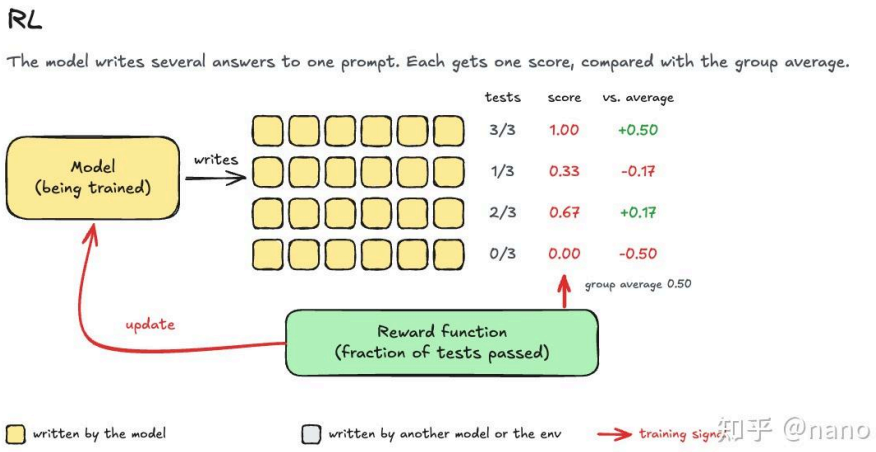
如果没有合适教师，但程序能可靠判定任务结果，GRPO 是一个可选入口。

同一个 prompt 采样一组回答，例如 8 条，各自得到奖励。在组内标准化时，用奖励减去组均值、再按标准差缩放，得到相对优势，模型据此提高组内较好回答的概率。它利用了分数差异，只说“看组内排名”会丢掉这层机制。

组内比较减少了对独立价值网络的需要，但不会替你解决判分问题。奖励函数如果只识别漂亮格式，模型就有机会学会满足格式而绕开任务。

多轮 rollout 的生成成本也要先算。假设 12 轮、每轮 800 token，一条约 9600 token；采样 8 条约 76800 token，还要执行 8 条轨迹里的环境动作。这是一个组的采样量，实际参数更新怎样组织还受批次与迭代设置影响。

单轮可判定的任务，可以先把奖励挂在最终答案，格式奖励只作辅助。多轮任务还需要看环境中的行为后果。



Hugging Face 原图：同一 prompt 多次采样，示例以测试通过比例打分，再与组均值比较。

```python
import re
from datasets import Dataset
from trl import GRPOConfig, GRPOTrainer

def extract_final(text):
    # 取最后一处 <answer>...</answer>；没有标签就退回整段文本
    found = re.findall(r"<answer>(.*?)</answer>", text, flags=re.S)
    return found[-1].strip() if
```

```
# 冒烟测试用：正式训练换成你自己的 prompt/answer 数据集
dataset = Dataset.from_dict({
    "prompt": ["计算 17 × 23，把结果写进 <answer> 标签。"],
    "answer": ["391"],
})

trainer = GRPOTrainer(
    model="Qwen/Qwen2.5-0.5B-Instruct",  # 正式训练换 SFT checkpoint
    reward_funcs=accuracy_reward,
    train_dataset=dataset,        # prompt 与 answer 两列
    args=GRPOConfig(
        num_generations=8,        # 每个 prompt 采一组
        scale_rewards="group",    # 组内标准化
        beta=0.04,                # KL 约束
        output_dir="out/grpo",
    ),
)
trainer.train()
```

这个只有一道乘法题的数据集用于冒烟检查，正式训练应替换为自己的任务。 `beta=0.04` 是示例选择的 KL 约束系数，不能当成所有版本的默认值。

训练之前先做四项排查：

1. 奖励函数能在一批样本上给出不同分数，避免全 0 或全 1。
2. 离线跑 100 条 rollout，看清分数分布。
3. 按失败原因记录奖励日志，让均值之外的故障可见。
4. 检查温度和组大小。温度太低，一组回答可能几乎相同。

reward 变成直线、loss 接近零时，先看组内方差。整组全对或全错，相对优势都会缺少区分度。可以换难度合适的 prompt；必要时暂加可靠的格式奖励让组内出现差别，再回到结果奖励。格式分不能长期替代任务成功。

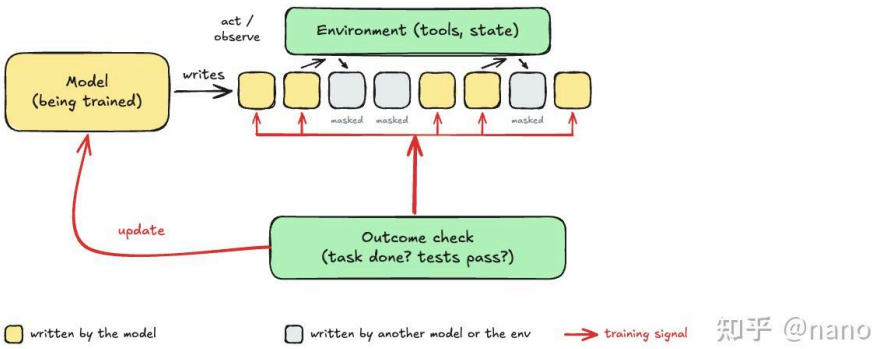
如果模型输出符合正则却答错，把判分推进到实际执行结果。提高 `num_generations` 后显存压力上升，则检查生成批次、组大小和 vLLM 的接法，具体增幅依赖配置。

GRPO 每条完成轨迹只给一个结果分数，定位不到具体哪一轮出了问题。要从稀疏反馈里学到稳定策略，通常需要更多采样，成本会随组大小与轨迹长度增加。

RL 环境：把动作的后果接回下一轮

当后续观察取决于前一步动作，环境就需要跨轮保留状态。导航、浏览网页、改代码再运行测试，都有这种连续性。

接法	状态特点	适用任务
tools	调用无状态、彼此独立	单次函数调用即可判断
environments	保留跨轮状态，动作改变后续观察	导航、浏览等连续交互



Hugging Face 原图：环境跨轮保留状态，工具观察不直接计模型损失，任务结果监督模型动作。

OpenEnv 提供环境协议，TRL 用 `environment_factory` 接入 GRPO。按本文采用的接口，执行过程分为五段：

1. factory 为每条 rollout 创建独立环境实例，状态不跨轨迹共享。
2. 训练循环调用 `reset(**reset_kwargs)` 初始化。数据集已提供 prompt 时，返回 `None` 避免重复追加；由环境出题时，则返回初始 observation。
3. 环境公开方法成为可调用工具，用 docstring 和 `Args:` 说明参数，这些描述会进入工具 schema。
4. Trainer 解析 tool call、执行方法、回填 observation，持续到模型停止调用工具或达到 `max_completion_length` 。
5. rollout 结束后， `reward_funcs(environments=...)` 从各环境读取 `env.reward` ，为每条轨迹提供一个标量。

线上已有 Harness 时，要先确认它的上下文、工具和停止逻辑能怎样映射到环境。训练环境若丢掉这些逻辑，训练出来的策略仍可能对不上部署。

Echo 示例只保留一个工具，用它看清 reset、状态与奖励的接线。

```python
from datasets import Dataset
from trl import GRPOConfig, GRPOTrainer

ECHO_TASK = "请调用 echo 工具两次，每次原样复述：训练环境会记住你的每一步。"

class EchoEnv:
    """最小环境：一个 echo 工具，记录调用次数。"""

    def __init__(self):
        self.turns = 0
        self.hits = 0
        self.reward = 0.0

    def reset(self, **kwargs):
        self.turns = 0
        self.hits = 0
        self.reward = 0.0
        # prompt 由 train_dataset 提供，返回 None 避免把任务再追加一遍
        return None

    def echo(self, text: str) -> str:
        """原样返回输入文本，并累计命中次数。

        Args:
            text: 要复述的文本。
        """
        self.turns += 1
```

```
        return f"第 {self.turns} 轮，收到。"

def echo_reward(environments, **kwargs):
    # rollout 结束后读一次，整条轨迹共用一个标量
    return [env.reward for env in environments]

def environment_factory():
    return EchoEnv()          # 每次 rollout 新建，状态不共享

train_dataset = Dataset.from_dict({"prompt": [ECHO_TASK]})

trainer = GRPOTrainer(
    model="Qwen/Qwen2.5-0.5B-Instruct",
    reward_funcs=echo_reward,
    train_dataset=train_dataset,
    environment_factory=environment_factory,
    args=GRPOConfig(
        num_generations=8,
        max_completion_length=1024,
        output_dir="out/rl-env",
    ),
)
trainer.train()
```

`env.reward` 在工具执行中更新，奖励函数在轨迹结束时读取。它没有保证模型一定恰好调用两次，正式任务的奖励还需准确覆盖你真正要的行为。

这里尤其要做五项检查：

1. factory 能在子进程独立重建，不依赖模块级可变状态。
2. reset 返回的初始信息符合线上约定；数据集有 prompt 时避免再追加一遍。
3. 每个 episode 同时受生成长度与工具超时约束。
4. `reward_funcs` 接收到环境实例，分数随轨迹变化，reset 会清零奖励。
5. 手动执行 3 条完整 episode，打印每次工具调用和 observation。

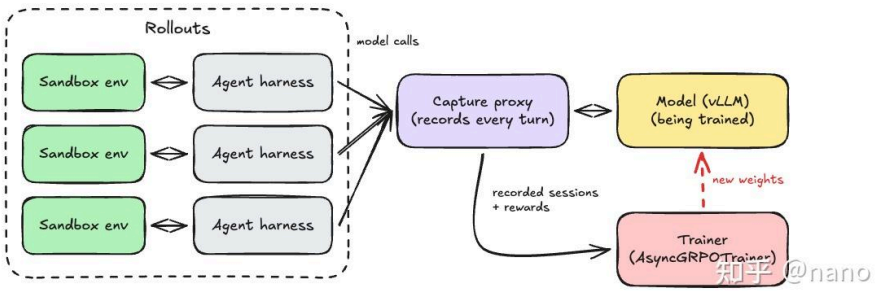
多个 rollout 领到同一道题时，检查是否共享模块级任务队列；可按 worker 分片，让每个环境独立领题。episode 一直不结束时，检查重复调用，为工具加超时、为环境加调用次数限制与明确终止反馈，并确认 Harness 如何处理。

奖励读不到或串到下一条轨迹时，先查 `environments` 参数，再查 reset 是否清零 `env.reward` 。

环境把行为后果带进训练，也延长了 episode。工具慢、沙箱慢时，环境执行会成为 rollout 的瓶颈，GPU 利用率本身不能解释全部训练耗时。

Harness 与异步：何时值得把生成和训练拆开？

当 rollout 耗时超过训练 step，同步流程会留下等待。 `AsyncGRPOTrainer` 把生成与训练解耦，让后台进程或独立资源生成轨迹，训练端消费返回的数据。



Hugging Face 原图：沙箱中的 Harness 执行完整 run，代理记录每次模型调用，轨迹和奖励送往训练端。

接入前先问：线上 Harness 的循环由谁管理？

1. 能拆成环境时，使用白盒 `environment_factory` 。TRL 管理训练中的循环，环境提供 reset、工具与奖励；沙箱地址从环境变量等配置传入，生成边界遵循训练循环。
2. Harness 自己管理循环、难以拆开时，可用 OpenEnv 的 `HarnessRolloutWorker` 包装 session factory，再交给 `AsyncGRPOTrainer(rollout_worker=...)` 。工具循环和停止条件留在 Harness，训练侧接收轨迹。

异步接入前，先看 rollout 与训练 step 的时间比。低于 1 时，优先确认异步是否真能消除等待；大于 1 时才有更明确的解耦动机。这个比值是工程判断线索，仍需结合实际流水线测量。

worker 与训练侧的依赖要对齐，异步文档要求 `vllm` 、 `transformers>=5.2.0` 。还要留意 vLLM 与 Transformers 的依赖约束，以对应版本安装说明为准。

容易卡住的地方有三类：

1. 序列化。factory、工具和奖励函数要能送到 worker。闭包捕获 HTTP client、线程锁或数据库连接，可能在启动时报 `TypeError: cannot pickle '_thread.lock' object` 。连接应在 worker 内重建。
2. 资源位置。后台 rollout worker 在 `CUDA_VISIBLE_DEVICES=""` 下运行，其中的 GPU 奖励模型可能回退到 CPU。奖励函数宜保持轻量；确需 GPU 评分，可放到独立服务，worker 经 HTTP 调用。模型生成端由推理服务承担，不能混淆两类资源。
3. 分布式限制。文档当前只支持 FSDP2，其他并行方式需要另查支持情况。

扩容前保留五项检查：

1. 单机单 worker 先跑一条真实任务，再增加 worker。
2. worker 独立重建环境和沙箱连接。
3. 每条 rollout 记录轮数、耗时与停止原因。
4. 核对 worker 与训练侧的 vLLM、Transformers 版本。
5. 分开监控生成和训练耗时。

启动即 pickle 错误，查 factory 捕获对象；rollout 长期 pending，查沙箱可达性与工具超时。reward 上升、loss 没有同步改善时，还要检查异步采样滞后，可尝试降低 `max_staleness` 或减少 worker，再结合任务指标判断。

异步增加了生成、环境和训练三处的日志与故障面。没有人维护环境与沙箱，即使 GPU 账单可接受，也很难稳定运行这一层。

真正动手前，先选入口和退出条件

现在手里的条件	优先

成功轨迹不足，有更强模型	蒸馏	教师资源与额外前向成本
任务能可靠程序判分	GRPO	稀疏奖励，需要采样
后续观察依赖先前动作	RL 环境	环境执行与状态管理
要接完整线上 Agent，rollout 更慢且有人维护沙箱	AsyncGRPO	异步滞后与系统复杂度

五个位置可以分别验证链路：SFT 教正确动作，蒸馏给逐 token 分布，GRPO 给结果反馈，环境补上动作后果，Harness 接住完整运行方式。选择入口，要同时看数据、判分手段与部署形态。

已有正确走法、某个动作没学全时，可以先用 SFT 加多轮评测；模型自己生成的前缀持续偏离，则考虑 on-policy 信号。能程序判分的任务可先试 GRPO，需要连续状态时再接环境。异步这层还要额外满足效率收益与维护能力。

GRPO 冷启动要盯组内差异。全对或全错都难产生有效的相对优势，先调 prompt 难度，必要时用辅助格式分拉开差别，然后检查任务结果是否真的改善。

也给训练留一条退出条件：训练后仍在第 7 轮打转，先回查回填、截断与评测范围。任务记录若被裁成单轮，你很可能仍看不见训练要解决的故障。

有四种情况，后训练可以暂缓：数据没有成功轨迹，先补路径；结果只能靠人看，先做可靠评测或判分；Harness 每天改且没人负责沙箱，先稳定接口；团队抽不出一个人连续两周盯训练和评测，先把推理与 prompt 调到可判断的状态。这里的两周是投入建议，具体周期由项目决定。

技术负责人至少要能说清楚谁维护环境和沙箱。这个责任没落实，环境训练与异步扩容就缺少前提；独立单轮 GRPO 可以按自己的条件评估。

准备开始时，可以先用 0.5B 到 1B 的小模型验证前四类接线，第五类等维护能力到位再接。小模型有助于检查数据、环境与奖励，能省去多少调试时间要看项目。

明天就要实施的话，按以下顺序留下记录：

1. 用线上 Harness 跑现成 checkpoint，记录 `turns` 、 `repeated_calls` 、 `stop_reason` ，先定位故障。
2. 收集几十条成功轨迹，跑一次 SFT，对比完整多轮指标。
3. SFT 不足时，再分辨缺的是教师分布信号，还是结果奖励。
4. 测量 rollout 与 step 的耗时，有明确收益且有人维护沙箱，再决定异步。

文中第 7 轮、12 轮、每轮 800 token、8 条采样都是说明用的假设，没有 benchmark 含义。代码按 TRL v1.17.1 接口整理，未执行 GPU 训练；其中 eval Harness 需要对接你自己的项目。SFT 默认学习率为 2e-5，蒸馏与 GRPO 为 1e-6，先以对应版本默认值确认链路，再结合曲线和任务指标调整。

回到开头的循环，下一步先回放工具结果如何进入上下文。只有在同一套多轮评测里，看见 Agent 读完 `config.yaml` 后能采取正确的下一步，才有依据判断这次训练解决了什么。

参考与延伸阅读

- Hugging Face · Train Agents
- TRL SFT Trainer
- TRL Distillation Trainer
- TRL GRPO Trainer
- TRL Async GRPO Trainer
- TRL 与 OpenEnv 集成

- 本文的微信公众号版本

发布于 2026-11-02 15:20 · 北京 · 信息来源于 官方网站 · 包含 AI 辅助创作 作者对内容负责

人工智能 强化学习 (Reinforcement Learning) 智能体



HR赫莲娜京东自营旗舰店

品质之选，速来围观！ 立即购买

的广告

理性发言，友善互动



还没有评论，发表第一个评论吧

推荐阅读



EPICENTER Major - 第四日回顾

十贰十二 发表于DogeG...



TI8第四日中国选手6大精彩操作秀：肉山巢穴中的希望之光

电竞GO 发表于刀圈二三事

Te和Ti看待世界的区别

一、我对思维“T”的理解——“分析与建构”（T就是“思考分析”，更全面一点就是“分析与建构”，这是Te和Ti都具备的） T的内涵：T似乎很多时候都诉诸于“事物本身的合理”，即“合乎逻辑...

長安

GUI Agent

TV GUI Agent 训练 - SFT篇

iloveacm